

Solution Requirements

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Input Collection	The system shall provide a web form collecting 10 weather/rainfall readings (Temperature, Humidity, Cloud Cover, Annual, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, AvgJune, Sub) with required-field validation.	High
2	Routing	The system shall expose a GET /predict route to display the form and a POST /predict route to process submissions and return a result page.	High
3	Feature Scaling	The system shall transform submitted values using the same fitted scaler (transform.save) used during model training before prediction.	High
4	Prediction	The system shall load the trained classifier	High

		(floods.save) via Joblib and return a binary flood / no-flood prediction.	
5	Result Presentation	The system shall render a dedicated result page (chance.html / no_chance.html) with an animated risk gauge reflecting the prediction.	Medium
6	Theming	The system shall allow users to toggle between light and dark themes, persisting the choice across visits.	Low
7	Navigation	The system shall provide clear navigation between the home page, the prediction form, and back from the result page.	Medium
8	Other	The system shall degrade gracefully with a clear error when required fields are missing from the submitted form.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Prediction response should return quickly after form submission since the model is small and inference is CPU-only.	Result page renders in under 1 second on local run
2	Scalability	The app should handle	Runs under Gunicorn with

		multiple concurrent users when deployed with a production WSGI server.	multiple workers without errors
3	Reliability & Availability	The app should not crash on malformed or missing form input.	Missing field returns a handled 400 response, not a server crash
4	Usability & Accessibility	The UI should be readable in both bright and dark environments and usable on mobile.	Light/dark theme + responsive layout down to 360px width
5	Maintainability	Code should be organized into clear routes, templates, and static assets for easy updates.	Flask app.py, templates/, static/ separated per Flask convention

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
6	Portability	The app should run identically on a local machine and a cloud host.	Runs unchanged via gunicorn on Render